



JOBSHEET PRAKTIKUM BASIS DATA

**Jurusan Teknologi Informasi
POLITEKNIK NEGERI MALANG
2025/2026 Genap**

PERTEMUAN 13 Sub-Query

Team Teaching:

Prof. Ir. Yan Watequlis Syaifudin, ST., M.MT., Ph.D.

Candra Bella Vista, S.Kom., MT.

Moch Zawaruddin Abdullah, S.ST., M.Kom.

Yoppy Yunhasnawa, S.ST., M.Sc.

Ridwan Rismanto, S.ST., M.Kom., Ph.D.

Elok Nur Hamdana, S.T., M.T

M. Hasyim ratsanjani, S.Kom., M.Kom.



Mata Kuliah	: Praktikum Basis Data
Program Studi	: D4 – Teknik Informatika / D4 – Sistem Informasi Bisnis
Semester	: 4 (empat)
Pertemuan ke-	: 13 (tiga belas)

JOBSHEET 13

SUB-QUERY

Tujuan Pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

- Memahami konsep Sub-Query dan penggunaannya dalam SQL
- Menggunakan operator EXISTS dan NOT EXISTS untuk mengecek keberadaan data
- Menggunakan operator ANY dan ALL untuk perbandingan terhadap himpunan nilai
- Membuat Sub-Query pada klausa FROM (Derived Table / Inline View)
- Membuat Correlated Sub-Query yang merujuk kolom dari outer query

Materi Pengantar

1. Review Sub-Query

Sub-query adalah query SELECT yang ditulis di dalam query lain. Disebut juga Nested Query atau Inner Query. Sub-query dieksekusi terlebih dahulu, hasilnya digunakan oleh outer query.

Sudah dipelajari di Pertemuan 11: Sub-query dengan IN (tes keanggotaan), sub-query dengan operator perbandingan, sub-query di SELECT, dan Fungsi Agregasi (MIN, MAX, AVG, COUNT, SUM).

Pertemuan 13 ini akan membahas topik Sub-Query lanjutan: EXISTS/NOT EXISTS, ANY/ALL, Derived Table, dan Correlated Sub-Query.

2. EXISTS dan NOT EXISTS

2.1 EXISTS

EXISTS mengembalikan TRUE jika sub-query menghasilkan minimal 1 baris. Tidak peduli isi datanya — yang penting ada hasil.

Format:

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS
  (SELECT 1 FROM table2 WHERE kondisi);
```

Tip: SELECT 1 di dalam EXISTS lebih efisien karena DBMS tidak perlu mengambil data kolom sebenarnya, cukup cek keberadaan baris.

Cara kerja EXISTS:

- Outer query mengambil satu baris dari tabel utama
- Sub-query dijalankan dengan kondisi yang mengacu ke baris tersebut
- Jika sub-query menghasilkan ≥ 1 baris, kondisi bernilai TRUE dan baris disertakan
- Jika sub-query kosong, kondisi bernilai FALSE dan baris dilewati
- Proses diulang untuk setiap baris (bersifat correlated)

2.2 NOT EXISTS

NOT EXISTS mengembalikan TRUE jika sub-query tidak menghasilkan baris sama sekali. NOT EXISTS merupakan kebalikan dari EXISTS.

Format:

```
SELECT column_name(s)
FROM table_name
WHERE NOT EXISTS
  (SELECT 1 FROM table2 WHERE kondisi);
```

Peringatan: Hati-hati NOT IN dengan NULL! Jika sub-query menghasilkan satu nilai NULL saja, seluruh hasil NOT IN menjadi kosong. Gunakan NOT EXISTS sebagai pengganti yang lebih aman.

Perbandingan IN vs EXISTS vs NOT IN vs NOT EXISTS:

Aspek	IN	EXISTS	NOT IN	NOT EXISTS
Keamanan NULL	Ada risiko	Aman	Berbahaya	Aman
Performa	Sedang	Cepat	Lambat	Cepat
Correlated	Tidak	Ya	Tidak	Ya
Kembalikan	Set nilai	Boolean	Set nilai	Boolean

3. ANY dan ALL

Operator ANY dan ALL digunakan bersama operator perbandingan ($=$, $<>$, $<$, $>$, $<=$, $>=$) untuk membandingkan satu nilai dengan sekumpulan nilai dari sub-query.

3.1 ANY (atau SOME)

ANY mengembalikan TRUE jika kondisi terpenuhi untuk MINIMAL SATU nilai dalam hasil sub-query.

```
SELECT nim, nilai_angka
```

```
FROM nilai
WHERE nilai_angka > ANY (
    SELECT nilai_angka FROM nilai
    WHERE kode_matakuliah = 'ASD'
); -- TRUE jika > salah satu nilai saja
```

3.2 ALL

ALL mengembalikan TRUE jika kondisi terpenuhi untuk SEMUA nilai dalam hasil sub-query.

```
SELECT nim, nilai_angka
FROM nilai
WHERE nilai_angka > ALL (
    SELECT nilai_angka FROM nilai
    WHERE kode_matakuliah = 'ASD'
); -- TRUE jika > SEMUA nilai dalam set
```

Ekuivalensi operator ANY dan ALL:

Operator	Ekuivalen	Keterangan
= ANY	IN	Sama dengan salah satu nilai
<> ALL	NOT IN	Tidak sama dengan semua nilai (aman dari NULL)
> ANY	> MIN(...)	Lebih besar dari nilai terkecil
> ALL	> MAX(...)	Lebih besar dari nilai terbesar
< ANY	< MAX(...)	Lebih kecil dari nilai terbesar
< ALL	< MIN(...)	Lebih kecil dari nilai terkecil

4. Sub-Query di FROM (Derived Table / Inline View)

Sub-query di klausa FROM menghasilkan tabel sementara (virtual) yang diperlakukan seperti tabel biasa. Sub-query ini wajib diberi ALIAS.

Penting: Tanpa alias, query akan error. Alias diberikan menggunakan kata kunci AS setelah tanda kurung tutup sub-query.

Kegunaan utama:

- Melakukan agregasi berlapis, misalnya GROUP BY kemudian filter menggunakan WHERE
- Menyederhanakan query yang kompleks dengan membagi menjadi beberapa bagian
- Dapat digunakan sebagai alternatif CTE (WITH clause) untuk keterbacaan lebih baik

```
SELECT ringkasan.kode_matakuliah, ringkasan.rata_nilai, ringkasan.jumlah_mhs
FROM ( -- sub-query di FROM, hasil jadi tabel sementara
    SELECT kode_matakuliah,
        AVG(nilai_angka) AS rata_nilai,
        COUNT(*) AS jumlah_mhs
    FROM nilai
    GROUP BY kode_matakuliah
) AS ringkasan -- alias WAJIB diberikan!
WHERE ringkasan.rata_nilai > 60
ORDER BY ringkasan.rata_nilai DESC;
```

5. Correlated Sub-Query

Correlated Sub-Query adalah sub-query yang merujuk kolom dari outer query. Berbeda dengan sub-query biasa yang dieksekusi sekali, correlated sub-query dieksekusi SATU KALI PER BARIS dari outer query.

Perbedaan Non-Correlated dan Correlated:

Non-Correlated (dieksekusi SEKALI)	Correlated (dieksekusi PER BARIS)
<pre>SELECT nim, nilai_huruf FROM nilai WHERE nilai_huruf IN (SELECT MIN(nilai_huruf) FROM nilai);</pre>	<pre>SELECT nim, kode_matakuliah, nilai_angka FROM nilai n1 WHERE nilai_angka = (SELECT MAX(nilai_angka) FROM nilai n2 WHERE n2.kode_matakuliah = n1.kode_matakuliah -- korelasi!);</pre>

Perhatian Performa: Jika tabel memiliki N baris, sub-query dieksekusi N kali. Pada data besar, ini bisa lambat. Pertimbangkan JOIN atau window function sebagai alternatif.

Kegiatan Praktikum

Praktikum 1: Persiapan Database

Pada bagian ini, Anda akan mempersiapkan database akademik yang akan digunakan selama praktikum.

- Buka DBeaver dan buat database baru, beri nama **dbakademik**.
- Buat SQL Script baru, kemudian salin isi file **prak13_akademik.sql** yang disertakan bersama jobsheet ini.
- Execute script tersebut untuk membuat database beserta tabel-tabelnya.
- Refresh Database Navigator dan pastikan database akademik telah berisi tabel-tabel berikut:

Tabel	Primary Key	Kolom
mahasiswa	nim	nim (INT), nama (VARCHAR)
matakuliah	kode	kode (VARCHAR), nama (VARCHAR), sks (INT), wajib (TINYINT)
nilai	id	id (INT), nim (INT FK), kode_matakuliah (VARCHAR FK), nilai_huruf (CHAR), nilai_angka (INT)

- Lakukan query SELECT sederhana pada setiap tabel untuk memverifikasi bahwa data telah berhasil dimasukkan.

Praktikum 2: EXISTS

Operator EXISTS digunakan untuk mengecek apakah sub-query menghasilkan minimal satu baris. Pada praktikum ini, kita akan menampilkan data mahasiswa berdasarkan keberadaan data pada tabel nilai.

Langkah 1: Tampilkan mahasiswa yang pernah mendapat nilai huruf 'A'

Identifikasi kebutuhan query:

- Nama mahasiswa ada di tabel mahasiswa
- Data nilai ada di tabel nilai
- Kondisi: EXISTS baris di tabel nilai yang ber-NIM sama dan nilai_huruf = 'A'

```
SELECT m.nim, m.nama
FROM mahasiswa m
WHERE EXISTS (
  SELECT 1
  FROM nilai n
  WHERE n.nim = m.nim
  AND n.nilai_huruf = 'A'
);
```

- Execute query dan perhatikan hasilnya. Mahasiswa mana saja yang pernah mendapat nilai A?
- Perhatikan bagaimana kondisi `n.nim = m.nim` menghubungkan sub-query dengan outer query (correlated).

Langkah 2: Bandingkan dengan pendekatan menggunakan IN

Tuliskan query yang setara menggunakan IN untuk membandingkan:

```
SELECT nim, nama
FROM mahasiswa
WHERE nim IN (
  SELECT nim
  FROM nilai
  WHERE nilai_huruf = 'A'
);
```

- Execute query dan bandingkan hasilnya dengan query EXISTS sebelumnya.
- Apakah hasilnya sama? Catat perbedaan cara kerja keduanya.

Praktikum 3: NOT EXISTS

NOT EXISTS digunakan untuk menemukan data yang tidak memiliki pasangan di tabel lain. Pada praktikum ini, kita akan menemukan mahasiswa yang belum memiliki nilai.

Langkah 1: Tampilkan mahasiswa yang belum memiliki nilai sama sekali

```
SELECT m.nim, m.nama
FROM mahasiswa m
WHERE NOT EXISTS (
  SELECT 1
  FROM nilai n
  WHERE n.nim = m.nim
);
```

```
);
```

- Execute query. Mahasiswa mana yang belum memiliki nilai?
- Cek dengan melihat data di tabel nilai dan pastikan hasil query sesuai.

Langkah 2: Tampilkan mata kuliah yang belum ada mahasiswa yang mengambilnya

Gunakan NOT EXISTS untuk menemukan mata kuliah yang tidak memiliki entri di tabel nilai:

```
SELECT mk.kode, mk.nama
FROM matakuliah mk
WHERE NOT EXISTS (
    SELECT 1
    FROM nilai n
    WHERE n.kode_matakuliah = mk.kode
);
```

- Execute query dan perhatikan mata kuliah mana yang belum ada nilainya.
- Coba bandingkan dengan menggunakan NOT IN dan perhatikan apakah hasilnya sama.

Catatan penting: Jika ada nilai NULL pada kolom kode_matakuliah di tabel nilai, pendekatan NOT IN akan menghasilkan result set kosong meskipun seharusnya ada data. Itulah mengapa NOT EXISTS lebih aman digunakan.

Praktikum 4: ANY dan ALL

Operator ANY dan ALL memungkinkan perbandingan terhadap sekumpulan nilai yang dihasilkan sub-query.

Langkah 1: Gunakan ANY untuk perbandingan nilai

Tampilkan nim dan nilai_angka mahasiswa yang nilai angkanya lebih besar dari minimal satu nilai angka di mata kuliah ASD:

```
SELECT nim, kode_matakuliah, nilai_angka
FROM nilai
WHERE nilai_angka > ANY (
    SELECT nilai_angka
    FROM nilai
    WHERE kode_matakuliah = 'ASD'
);
```

- Execute query dan amati hasilnya.
- Bandingkan dengan query menggunakan > MIN(...) untuk memverifikasi ekuivalensinya:

```
SELECT nim, kode_matakuliah, nilai_angka
FROM nilai
WHERE nilai_angka > (
    SELECT MIN(nilai_angka)
    FROM nilai
    WHERE kode_matakuliah = 'ASD'
);
```

Langkah 2: Gunakan ALL untuk perbandingan nilai

Tampilkan nim dan nilai_angka mahasiswa yang nilai angkanya lebih besar dari SEMUA nilai angka di mata kuliah ASD:

```
SELECT nim, kode_matakuliah, nilai_angka
FROM nilai
WHERE nilai_angka > ALL (
  SELECT nilai_angka
  FROM nilai
  WHERE kode_matakuliah = 'ASD'
);
```

- Execute query. Perhatikan perbedaan jumlah baris hasil dibanding query ANY sebelumnya.
- Mengapa hasilnya berbeda? Berapa nilai angka tertinggi di mata kuliah ASD? Baris mana saja yang memenuhi kondisi > semua nilai tersebut?

Praktikum 5: Sub-Query di FROM (Derived Table)

Sub-query di klausa FROM menghasilkan tabel sementara yang bisa langsung digunakan oleh outer query. Teknik ini sangat berguna untuk melakukan filter atau operasi setelah agregasi.

Langkah 1: Filter hasil agregasi menggunakan Derived Table

Tampilkan kode mata kuliah beserta rata-rata nilai angka dan jumlah mahasiswanya, hanya untuk mata kuliah yang rata-rata nilainya di atas 75:

```
SELECT ringkasan.kode_matakuliah,
       ringkasan.rata_nilai,
       ringkasan.jumlah_mhs
FROM (
  SELECT kode_matakuliah,
         AVG(nilai_angka) AS rata_nilai,
         COUNT(*) AS jumlah_mhs
  FROM nilai
  GROUP BY kode_matakuliah
) AS ringkasan
WHERE ringkasan.rata_nilai > 75
ORDER BY ringkasan.rata_nilai DESC;
```

- Execute query dan perhatikan hasilnya.
- Coba hilangkan alias ringkasan setelah tanda kurung tutup sub-query. Apa yang terjadi?
- Mengapa filter rata_nilai > 75 tidak bisa langsung ditulis menggunakan HAVING pada query dalam sub-query?

Langkah 2: Gabungkan Derived Table dengan tabel lain

Tampilkan nama mata kuliah (dari tabel matakuliah) beserta rata-rata nilai angkanya, diurutkan dari rata-rata tertinggi:

```
SELECT mk.nama AS nama_matakuliah,
       r.rata_nilai,
       r.jumlah_mhs
FROM (
  SELECT kode_matakuliah,
         AVG(nilai_angka) AS rata_nilai,
         COUNT(*) AS jumlah_mhs
  FROM nilai
  GROUP BY kode_matakuliah
) AS r
JOIN matakuliah mk ON mk.kode = r.kode_matakuliah
```

```
ORDER BY r.rata_nilai DESC;
```

- Execute query dan perhatikan cara derived table dapat di-JOIN dengan tabel lain seperti tabel biasa.

Praktikum 6: Correlated Sub-Query

Correlated Sub-Query adalah sub-query yang merujuk kolom dari outer query, sehingga dieksekusi ulang untuk setiap baris outer query.

Langkah 1: Tampilkan nilai tertinggi per mata kuliah

Tampilkan nim, kode_matakuliah, dan nilai_angka untuk baris yang nilai angkanya merupakan nilai tertinggi di mata kuliah yang sama:

```
SELECT n1.nim, n1.kode_matakuliah, n1.nilai_angka
FROM nilai n1
WHERE n1.nilai_angka = (
    SELECT MAX(n2.nilai_angka)
    FROM nilai n2
    WHERE n2.kode_matakuliah = n1.kode_matakuliah -- korelasi
)
ORDER BY n1.kode_matakuliah;
```

- Execute query. Perhatikan bagaimana n1.kode_matakuliah pada sub-query merujuk ke outer query.
- Apakah ada mata kuliah yang memiliki lebih dari satu pemenang (nilai tertinggi yang sama)?

Langkah 2: Tampilkan nilai tertinggi setiap mahasiswa bersama informasi nilai maksimum mata kuliahnya

Gunakan correlated sub-query di klausa SELECT untuk menampilkan nilai maksimum per mata kuliah sebagai kolom tambahan:

```
SELECT n.nim,
       n.kode_matakuliah,
       n.nilai_angka,
       (SELECT MAX(nilai_angka)
        FROM nilai n2
        WHERE n2.kode_matakuliah = n.kode_matakuliah) AS nilai_max_mk
FROM nilai n
ORDER BY n.kode_matakuliah, n.nilai_angka DESC;
```

- Execute query. Kolom nilai_max_mk menampilkan nilai maksimum di setiap mata kuliah yang sama.
- Untuk setiap baris, apakah nilai_angka sama dengan nilai_max_mk? Baris mana yang merupakan peraih nilai tertinggi?

Langkah 3: Mahasiswa yang mengambil SEMUA mata kuliah wajib

Gunakan double NOT EXISTS untuk menemukan mahasiswa yang telah mengambil semua mata kuliah wajib (wajib = 1):

```
SELECT m.nim, m.nama
FROM mahasiswa m
```

```

WHERE NOT EXISTS (
  SELECT 1
  FROM matakuliah mk
  WHERE mk.wajib = 1
  AND NOT EXISTS (
    SELECT 1
    FROM nilai n
    WHERE n.nim = m.nim      -- korelasi luar
    AND n.kode_matakuliah = mk.kode -- korelasi dalam
  )
);

```

- Execute query. Logika query ini: tampilkan mahasiswa yang TIDAK ADA mata kuliah wajib yang BELUM diambilnya.
- Identifikasi manual dari data: mata kuliah wajib ada ASD, BD, dan PBO. Mahasiswa mana yang sudah mengambil ketiganya?
- Apakah hasil query sesuai dengan identifikasi manual Anda?

Tugas Praktikum

Kerjakan soal-soal berikut menggunakan database akademik yang telah disiapkan pada Praktikum 1. Sertakan screenshot hasil query pada laporan praktikum.

No.	Soal
1.	Tampilkan nim dan nama mahasiswa yang pernah mendapat nilai huruf 'A'. Gunakan EXISTS.
2.	Tampilkan nim dan nama mahasiswa yang TIDAK pernah mendapat nilai huruf 'A'. Gunakan NOT EXISTS.
3.	Tampilkan kode dan nama mata kuliah yang rata-rata nilai angkanya lebih dari 70. Gunakan sub-query di klausa FROM (derived table).
4.	Tampilkan nim, kode_matakuliah, dan nilai_angka untuk mahasiswa yang nilai angkanya lebih tinggi dari rata-rata nilai angka di mata kuliah yang sama. Gunakan correlated sub-query.
5.	Tampilkan nim dan nama mahasiswa yang telah mengambil SEMUA mata kuliah yang bersifat wajib (wajib = 1). Gunakan double NOT EXISTS.
6.	Tampilkan nim dan nama mahasiswa beserta jumlah mata kuliah yang telah diambil. Gunakan correlated sub-query di klausa SELECT.
7.	Tampilkan nama mata kuliah yang BELUM diambil oleh mahasiswa dengan nim = 171402. Gunakan NOT EXISTS atau NOT IN.
8.	Tampilkan nim dan nama mahasiswa yang memiliki nilai angka di atas nilai angka semua mahasiswa pada mata kuliah 'ML'. Gunakan ALL.

Referensi

- Dwi Puspitasari, S.Kom, "Buku Ajar Dasar Basis Data", Program Studi Manajemen Informatika Politeknik Negeri Malang, 2012.
- Fathansyah, "Basisdata Revisi Kedua", Bandung: Informatika, 2015.
- https://www.w3schools.com/sql/sql_exists.asp
- <https://dev.mysql.com/doc/refman/8.0/en/subqueries.html>
- <https://dev.mysql.com/doc/refman/8.0/en/any-in-some-subqueries.html>